

How to Do an Optimization Homework

A working method, step by step. IE 5311-001 (D01) Principles of Optimization, Fall 2026. Instructor: Ningji Wei.

This is the machine. The same seven moves every time, whatever the story is. Work through it with the sheet open beside you. After about five problems you will not need it any more.

The thing nobody tells you first

An optimization problem is always the same three questions wearing a costume:

1. What am I allowed to choose?
2. What am I trying to make big, or small?
3. What stops me from cheating?

Investment planning, shortest path, a warehouse network, Sudoku. All four of our homework problems are that. The story changes. The machine does not.

Step 1. Find the decision

Read the problem once for the story. Then read it again asking one question only: what could I change, if I were in charge?

The test that never fails: **if I handed you these numbers on paper, could you walk out and execute it?**

- "Put \$3,000 into the two-year deposit at the start of Year 1" — yes, executable, so it is a **decision**.
- "The two-year deposit pays 9%" — you cannot change that, so it is a **parameter**.
- "I end Year 5 with \$6,267.50" — that is what *happens*, not what you *choose*. A **result**, not a variable.

That last one is where most people lose marks. The quantity the problem asks you to maximize is usually **not** a variable. It is a consequence of your variables.

Problem	The decision is	Not
P1 Investment	dollars into each instrument at each epoch	the final balance
P2 Path	which arcs you walk on	the path length
P3 Network	units shipped on each lane	total cost
P4 Sudoku	which digit sits in which cell	"solving the puzzle"

Step 2. Index before you write anything

This is the class rule and it is not decoration. Never write x_1 , x_2 , x_3 .

Ask: what does this decision vary over? Count the answers. That is your number of subscripts.

- Varies over *time* and *which instrument*, so two things, so $x_{\{t,k\}}$
- Varies over *which arc*, so one thing (though an arc is a pair), so $x_{\{ij\}}$
- Varies over *row*, *column*, *digit*, so three, so $x_{\{r,c,v\}}$

Then name the set each subscript lives in, before the variable exists:

$T = \{1, \dots, 5\}$ epochs, indexed by t
 $K = \{1, 2, 3\}$ instrument terms, indexed by k

I assume this feels like extra paperwork the first time. It is not. The moment the professor says "now do it with 40 foods instead of 6", the indexed model needs zero edits. The x_1, x_2, x_3 model needs a rewrite.

Step 3. Fill the table before you write any math

Four blocks, in this order, always.

Block	What goes in it
Sets	with the index letter
Parameters	with units. c_i = cost per unit of food i , dollars per unit
Decision variables	with domain and units. $x_i \geq 0$ = units of food i per day, units per day
Then the model	objective, constraints, domains

The units rule is the cheapest error-catcher you own. If you cannot write the unit, you do not yet understand the quantity. And when you multiply c_i times x_i you should get dollars per day. If you get dollars per unit squared, something upstream is wrong.

Step 4. The objective is one line

It is always a sum over an index set.

$\min \sum_{i \in I} c_i x_i$

If a specific number appears in your objective, you have made a mistake. Numbers live in the parameter table. The objective mentions symbols only.

Step 5. Constraints, or English into mathematics

This is the part that feels like magic until you see that it is a lookup table. Go through the problem text sentence by sentence. Every sentence that restricts you becomes a row.

What the problem says	What you write
"at most", "no more than", "capacity of", "budget"	$\leq$
"at least", "minimum", "must cover", "required"	$\geq$
"exactly", "all of it", "must equal", "one per"	$=$
"each", "every", "for all"	the quantifier: for all i in I
"either / or", "if we open, then", "yes or no"	a binary variable
"what arrives must leave"	flow balance: $\text{out}(i) - \text{in}(i) = b_i$
"visit at most once"	degree constraint: ≤ 1

Three questions that sweep the problem for anything you missed:

1. **What is limited?** Capacity, supply, budget, hours. These give $\leq$ rows.
2. **What must be met exactly?** Demand, balance, "exactly one digit per cell". These give $=$ rows.
3. **What is a floor?** Nutrition minimums, service levels. These give $\geq$ rows.

Write each *family* once, with a quantifier:

$$\sum_{i \in I} a_{ij} x_i \leq b_j \quad \text{for all } j \in J$$

Not eleven separate rows with numbers inside them. One row, with a "for all".

Step 6. Domains go on their own line

$x_i \geq 0$, or $x_{ij} \in \{0,1\}$, or y_i integer. Kept separate from the structural constraints. The class wants them split, and it also lets you see at a glance whether you are holding a linear program or an integer program.

Step 7. Classify it, on the slide 8 to 10 taxonomy

Six questions, every model, no exceptions. This is where the deep-understanding marks live.

1. Constrained or unconstrained?
2. Variable type and size. Continuous, integer, binary? How many?
3. Objective type and size. Linear? Convex?
4. Constraint type and size. Linear? How many rows?
5. Parameters deterministic or stochastic?
6. How many decision stages? How many players?

Questions 5 and 6 are the ones people skip, and they are exactly what separated the three parts of Problem 3 from one another.

Before you touch a solver: four sanity checks

1. Check the arithmetic of the setup

In Problem 3, total supply is 120 and total demand is 120. Had those not matched, no model on earth would be feasible. Two minutes of adding saves an hour of debugging.

2. Find one feasible point by hand

Can you produce *any* legal answer? If you cannot, either the problem is genuinely infeasible or you have mis-translated a constraint.

3. Look for the free lunch

This is the one that bites hardest. Ask: can my model do something absurd and be rewarded for it?

In the longest-path problem the answer was yes. Maximize the sum of arc weights and the model happily returns a path **plus a pile of disconnected loops**, because every extra loop adds weight for free. Nothing in flow balance forbids it. That is why subtour elimination exists.

Under **minimization** with positive costs, a stray loop only adds cost, so it never appears and you never need those constraints. Under **maximization** they are mandatory. Same constraint set, opposite verdict, purely because the direction flipped.

Whenever you maximize, hunt for the free lunch before you solve.

4. Watch for the trivially-optimal trap

If your answer is "do nothing", or "one facility serves everybody", your instance may be too easy to demonstrate anything. That happened to us once and the data had to be replaced.

Then, and only then, solve

Solve a tiny version first. Three nodes, two foods, whatever. You can verify a tiny answer in your head. Scale up once the small one is right.

When the answer comes back, read it as an English sentence:

"Put all \$5,000 in the two-year deposit for Years 1 and 2, then roll everything into the Pioneer three-year certificate. Hold no idle cash. End with \$6,267.50."

If that sentence sounds insane, the model is wrong, not the world.

Then check the **shadow prices**. In Problem 1 the dual on the Year-1 balance came out at 1.2535, which is exactly the compounded multiplier of the winning plan. When the dual tells the same story as the primal, you have real confirmation. When it does not, go looking.

Writing it up

Same order as your thinking, every time.

1. Restate the problem in your own words, two or three sentences
2. Sets
3. Parameters, with units
4. Decision variables, with domain and units
5. Objective
6. Constraints, each family once, quantified
7. Domain restrictions
8. The slide 8 to 10 classification
9. Result, stated as a sentence, with the reasoning that confirms it

Code is a **check**, not the deliverable. Slide 6 puts 60 percent on method understanding. A perfect script with a hand-waved formulation scores badly. A correct formulation with no code at all scores well.

The four shapes you will keep meeting

Once you recognize these, most problems stop being new.

Shape	Signature	Where we met it
Allocation over time	a balance equation per period: what you hold equals what matured plus what you carried	P1 Investment
Network flow	$\text{out}(i) - \text{in}(i) = b_i$, with b positive at sources, negative at sinks, zero in the middle	P2 Paths, P3 Network
Assignment and covering	binary variables, and rows that sum to exactly 1	P4 Sudoku
Selection with a fixed charge	binary y_i for "open", continuous x_{ij} for "use", and the link $x_{ij} \leq y_i$ chaining them	facility location, class 01

On that last shape: write the link **disaggregated**, one row per pair. It gives a tighter LP bound than the lumped version, and we measured it. 45.0 against 28.0, with an integer optimum of 46.0.

The five mistakes that cost the most marks

1. **Numbers inside constraints.** Writing $3x_1 + 2x_2 \leq 18$ instead of $\sum_i a_{ij} x_i \leq b_j$ for all j . An automatic deduction in this class.
2. **Maximizing without checking for the free lunch.**
3. **Confusing the result with the decision.**
4. **Forgetting units**, then multiplying two things that should never have been multiplied.
5. **Skipping the classification.** It is worth marks and takes ninety seconds.

Where this recipe honestly stops

I will not pretend the whole thing is mechanical. Steps 3 through 7 are genuinely a checklist, and you can run them half asleep.

Steps 1 and 2 are judgment, and no recipe removes that. Deciding that the Problem 3 stochastic model splits into "factory shipments committed before you see the capacity" and "store deliveries chosen after" is a modelling choice. Reverse it and the whole problem collapses into a set of independent deterministic problems with nothing stochastic about them. The checklist does not make that call for you.

What the checklist does do is make sure that once you have made the call, nothing downstream gets fumbled. That is the eighty percent. The remaining twenty comes from working problems until the four shapes above become recognition rather than derivation.

Do five problems with this sheet open in front of you. By the sixth you will not need it.